SemanticShield: An ONNX-Native Runtime Assurance Layer for Network Intrusion Detection at the Edge

Muhammad Sikandar Hussain

Abstract—Machine-learning-based network intrusion detection systems (ML-NIDS) are increasingly exported to Open Neural Network Exchange (ONNX) format for cross-platform deployment on edge gateways. However, once exported, deep classifiers execute blindly: standard runtime engines lack mechanisms to validate input integrity, quantify prediction uncertainty, or detect representation shift at inference time. Rather than seeking an optimal out-of-distribution (OOD) ensemble, this paper introduces SemanticShield, an ONNX-native runtime assurance layer that surrounds deployed NIDS classifiers with deterministic structural validation and complementary statistical monitors operating directly on the computational graph.

By instrumenting the model export with an auxiliary penultimate-layer output (a 64-dimensional embedding), SemanticShield evaluates class-conditional Mahalanobis and cosine metrics alongside Maximum Softmax Probability (MSP) and a pre-inference input validator. On CSE-CIC-IDS2018 and NF-ToN-IoT-v2 under rigorous five-seed evaluation, our results characterize the operational coverage–false-alarm–resource trade-offs: (i) On real cross-domain distribution shift, class-conditional Mahalanobis distance achieves an AUROC of 0.9713 (AUPRC 0.9639, FPR@95 0.0616), while the composite drift score achieves an AUROC of 0.9486. (ii) Across nine distinct deployment-time failure modes, statistical OOD detectors fail completely on 100% zero-filled tampering (0.0% detection) and struggle on non-finite values (46.4%), whereas our deterministic validator achieves 100% interception. In aggregate, the full assurance layer intercepts 100% of zero-fill tampering and Gaussian noise, 96.5% of feature permutations, 90.3% of extreme outliers, and 79.3% of non-finite corruptions at an operational 1% FPR budget. (iii) We report a negative finding in telemetry reconciliation: resolving feature sets to standardized NetFlow schemas eliminates zero-filling but does not rescue cross-dataset generalization (multi-class F1 collapses from 0.766 to 0.057); an authoritative RFC-grounded audit reveals that 38.1% (8/21) of candidate features represent physically incompatible phenomena (reflecting deterministic concordance under formal RFC specifications). (iv) Under simulated edge resource constraints (profile R1 simulating a 1-thread execution environment with a 512 MB nominal memory target via ONNX Runtime thread execution controls on an x86_64 host), SemanticShield sustains 1,101 flows/s with 0.911 ms mean (1.340 ms p95) latency (adding a bounded +0.64 ms overhead) and flat 124.5 MB measured RSS. (v) Weight-only INT4 quantization preserves 97.8% of classification F1 with 80.7% size reduction, whereas static INT8 suffers severe degradation (−38.0% F1) caused by post-ReLU activation clipping.

Index Terms—Network intrusion detection, runtime assurance, ONNX Runtime, out-of-distribution detection, edge computing, telemetry standardization, model quantization.

M. S. Hussain is with CNIT/PNTLab Pisa, TECIP Institute, Scuola Superiore Sant'Anna, Pisa, Italy (e-mail: m.hussain@santannapisa.it).

I. Introduction

DEEP-learning-based network intrusion detection systems (ML-NIDS) achieve high benchmark accuracy on curated flow datasets [1], [2]. To bridge the gap between PyTorch training environments and production appliances, industry has broadly converged on the Open Neural Network Exchange (ONNX) format and ONNX Runtime [3]. ONNX decouples model training from execution, enabling high-throughput, hardware-accelerated scoring on edge gateways, switches, and industrial Internet-of-Things (IIoT) concentrators.

However, exporting a trained neural network into a static inference engine introduces a severe runtime assurance gap. Standard execution engines assume that incoming test vectors are independent, identically distributed, syntactically well-formed, and drawn from the training distribution. In live operational networks, this assumption fails continuously:

1) Silent distribution drift: Seasonal changes in benign traffic, emerging network protocols, and evolving adversary tactics cause feature distributions to diverge from training baselines [4], leading classifiers to emit confident, erroneous predictions.
2) Structural and telemetry corruption: Network flow extractors (e.g., CICFlowMeter, nProbe NetFlow, Zeek) implement divergent feature calculation logic. Protocol timeouts, parsing bugs, and telemetry schema mismatches inject missing fields, NaNs, and zero-filled vectors directly into model tensors [5], [6].
3) Adversarial and out-of-bounds evasion: Attackers craft perturbations designed to navigate decision boundaries, exploiting uncalibrated model uncertainty [7].

Prior literature has proposed out-of-distribution (OOD) detection techniques (e.g., Maximum Softmax Probability [8], Mahalanobis distance [9], Energy scoring [10]) to detect novel traffic. However, existing NIDS OOD approaches exhibit three critical shortcomings. First, they evaluate exclusively in unconstrained server environments, disregarding edge memory, CPU, and sub-millisecond latency bounds. Second, they evaluate isolated statistical anomaly scores on clean benchmark splits, ignoring malformed tensors, zero-fills, and non-finite features where

pure distance metrics fail completely. Third, they routinely require original training frameworks (PyTorch or TensorFlow) at runtime to hook intermediate activations, forfeiting the standalone efficiency of ONNX deployment.

In this work, we argue that securing edge ML-NIDS requires shifting focus from pursuing an elusive "universal OOD detector" to constructing a comprehensive, multi-layer runtime assurance layer. We investigate the following central research question:

> Central Research Question: Can an ONNX-native runtime assurance layer detect multiple deployment-time failure modes in ML-based NIDS while satisfying explicit false-alarm and edge-resource budgets?

To address this question systematically, we formulate three targeted sub-questions:

- RQ1 (Effectiveness & Operational Feasibility): Can an ONNX-native assurance layer intercept real cross-domain distribution shifts and structurally corrupted inputs under strict operational false-positive rate constraints (FPR $\leq 0.1\%$ and 1.0%)?
- RQ2 (Monitor Complementarity & Coverage): What is the specific contribution and complementarity of deterministic structural validation, embedding-space drift detection, and prediction uncertainty across distinct deployment failure modes?
- RQ3 (Edge Resource & Quantization Dynamics): What are the latency, memory footprint, and classification fidelity costs of runtime assurance when operating under controlled CPU/memory resource constraints and multi-precision quantization (FP16, static INT8, weight-only INT4)?

To answer these questions, we present SemanticShield, an ONNX-native runtime assurance framework designed specifically for resource-constrained edge gateways. SemanticShield instruments the ONNX computational graph with an auxiliary penultimate-layer output, coupling deterministic input validation with class-conditional embedding drift and confidence monitoring.

Contributions. The primary contributions of this paper are:

1) ONNX-native runtime assurance architecture: We design and implement a dual-output ONNX execution pipeline that executes structural validation, confidence thresholding, and class-conditional Mahalanobis/cosine drift scoring directly on the deployed graph without external framework dependencies.
2) Comprehensive failure-mode coverage & ablation: We evaluate SemanticShield across nine operational failure modes (real cross-domain shift, non-finite values, varying zero-fill ratios, feature permutations, sensor scale errors, and Gaussian noise). We prove that deterministic validation and statistical drift monitors are strictly complementary: statistical detectors score 0.0% on zero-filled inputs where validation achieves 100%, while validation is blind to clean distribution shift where Mahalanobis achieves 0.9713 AUROC.
3) Authoritative semantic feature audit & transfer negative result: Grounded in IETF RFC 7012 and RFC 793, we perform a source-verified audit of 21 candidate flow features between NetFlow v9 and CICFlowMeter, revealing that 38.1% represent physically incompatible quantities (codified strictly under formal RFC specifications). We report that standardizing schemas eliminates zero-filling but exposes severe generalization collapse (multi-class F1 collapses from 0.766 to 0.057), confirming that baseline transfer relied on zero-fill patterns as a spurious shortcut.
4) Simulated edge resource constraints & quantization dynamics: We benchmark SemanticShield across four controlled execution profiles (R0–R3) simulating 1–4 thread edge deployment profiles via ONNX Runtime thread execution controls on an x86_64 host without OS-level memory quota enforcement. On a 1-thread constrained profile (R1), SemanticShield sustains 1,101 flows/s with 0.911 ms mean (1.340 ms p95) latency (+0.64 ms overhead) and 124.5 MB measured RSS. Furthermore, empirical activation profiling explains why static INT8 suffers a catastrophic 38.0% F1 drop (activation outlier clipping), whereas weight-only INT4 preserves 97.8% of classification F1 with 80.7% compression.

The remainder of this paper is organized as follows. Section II positions SemanticShield within the literature. Section III specifies the proposed runtime assurance architecture. Section IV details the NetFlow semantic audit and transfer negative result. Section V presents the comprehensive empirical evaluation. Section VI discusses architectural insights, the INT8 paradox, and simulation limitations. Section VII examines threats to validity. Section VIII concludes the paper.

II. Related Work

Runtime assurance for edge NIDS intersects out-of-distribution detection, base-rate operational constraints, flow telemetry standards, cross-dataset transfer, model quantization, and ONNX deployment integrity. Table I provides a systematic comparative matrix.

A. OOD Detection Foundations in Machine Learning

Out-of-distribution detection identifies test inputs deviating from the training distribution, enabling systems to flag uncertain predictions. Hendrycks and Gimpel [8] established Maximum Softmax Probability (MSP). Lee et al. [9] proposed class-conditional Mahalanobis distance over intermediate layer representations, assuming class-conditional Gaussian distributions sharing a tied covariance matrix. Liu et al. [10] introduced unnormalized energy scoring. Liang et al. [16] developed ODIN using temperature scaling and input perturbations, while Sun et al. [17] proposed ReAct activation truncation. Sastry and Oore [18] computed Gram-matrix correlations. Gap: These methods were developed for continuous image

TABLE I: Systematic Prior-Work Comparison Matrix Across Telemetry, OOD, Precision, and Edge Evaluation Dimensions

System / Study	Deployment Domain	Flow Ingestion Format	Feature Reconciliation Methodology	Fixed-FPR Evaluated	Quantization Investigated	Edge Environment Evaluated	Root-Cause Error Diagnostics
Kitsune [11]	Edge / IoT Gateway	Raw Packet Damped Stats	None (Single stream)	No	No	Physical RPi 3B	No (Score only)
McLaughlin et al. [12]	Datacenter / Server	Tabular (CIC / NetFlow)	None (Uniform schemas)	Partial (FPR@95)	No	None (GPU Server)	Partial (Algorithm)
NetSight [13]	SDN / Switches	Packet History Digests	None (Homogeneous)	No	No	Switch Testbed	No (Path tracing)
Sarhan et al. [6]	Offline Benchmark	NetFlow v9 (nProbe)	Schema Projection (12/43)	No	No	None (Server)	No (Macro metrics)
Cantone et al. [14]	Offline Benchmark	CICFlowMeter (CIC/LycoS)	Common Column Subset	No	No	None (Server)	Partial (Artifacts)
Jajal et al. [5]	General DL Systems	DNN Computational Graphs	None (Operator lowering)	No	Partial (Bugs)	None (Host PC)	Yes (Graph bugs)
Yang et al. [15]	IoV / Vehicle Edge	CICFlowMeter / UNSW	Per-Dataset Feature Select	No	No	Node Emulation	No (Feature rank)
SemanticShield	Edge Gateway	Hybrid (NetFlow ↔ CIC)	Physical Audit (4 Tiers, RFC-grounded)	Yes ($\leq$ 0.1%/1.0%)	Yes (FP/INT8/INT4)	Yes (Simulated R0–R3)	Yes (Field & Tensor)

representations. Tabular flow telemetry is skewed, non-smooth, and bounded, requiring regularized covariance estimation and explicit structural validation to prevent degenerate distance computations.

B. Deep NIDS and Operational Base-Rate Constraints

Deep learning has achieved high accuracy on benchmark NIDS datasets [15], [19]–[21]. However, Axelsson [22] formalized the base-rate fallacy in intrusion detection: in high-throughput enterprise networks handling millions of flows per second, an FPR of even 1.0% yields tens of thousands of false alarms daily. Sommer and Paxson [23] emphasized that the "closed-world" assumption of offline benchmarks produces deceptive operational optimism. Consequently, detection metrics are operationally meaningless unless evaluated at strict false-alarm budgets (FPR $\leq 0.1\%$ or 1.0%).

C. Flow Telemetry Standards and Packet Aggregation

Network telemetry bridges physical packet streams and tabular feature representations [24]. Claise and Trammell [25] standardized the IP Flow Information Export (IPFIX) Information Model. Postel [26] codified TCP control bits and connection states. Handigol et al. [13] showed that packet history digests reconstruct flow lifecycles. Gap: Academic NIDS benchmarks frequently rely on ad-hoc feature extractors (such as CICFlowMeter) whose cumulative counters conflict with IPFIX definitions, creating latent incompatibilities when models encounter live enterprise telemetry.

D. Cross-Dataset Generalization Collapse

Cross-dataset transfer in NIDS exhibits persistent failure. Pontes et al. [27] demonstrated that while binary transfer is moderate, multi-class attack classification collapses to F1 < 0.10. Cantone et al. [14] confirmed that models memorize dataset-specific artifacts. Sarhan et al. [6] introduced NetFlow-standardized datasets to eliminate syntactic discrepancies. Gap: Existing studies attribute transfer collapse solely to statistical covariate shift. Our work demonstrates that physical semantic mismatch between extractors is a primary latent cause.

E. Edge Machine Learning and Model Quantization

Deploying deep models to embedded gateways requires compression [28]–[31]. Ahn et al. [32] evaluated INT8 and FP16 inference across x86 and ARM platforms, reporting that INT8 reduces model sizes by $4\times$ with negligible accuracy degradation in convolutional vision models. Gap: Quantization literature focuses on image and speech data with bounded Gaussian-like activations. In tabular flow telemetry, extreme dynamic ranges (0 to $> 10^9$) cause symmetric INT8 calibration to saturate, inducing catastrophic activation clipping that our empirical profiling exposes.

F. ONNX Runtime Interoperability and Deployment Integrity

ONNX and ONNX Runtime [3] provide cross-platform inference execution. However, Jajal et al. [5] analyzed 200 ONNX converter defects, revealing that 75% stem from node conversion errors and 33% execute silently without throwing runtime exceptions. Strategic Positioning: SemanticShield unifies these domains by providing an ONNX-native assurance layer that evaluates detection under fixed operational FPR constraints ($\leq$ 0.1% and 1.0%), eliminates incompatible features via RFC-grounded auditing, identifies activation clipping mechanisms, and executes sub-millisecond inference directly on the computational graph.

III. Proposed Method: SemanticShield

A. Dual-Output Computational Graph Architecture

Fig. 1 illustrates the SemanticShield architecture. To eliminate external deep-learning framework dependencies during production inference, the trained classifier (ThreatMLP: $d \to 256 \to 128 \to 64 \to C$, where $d = 13$ standardized flow features and $C = 15$ attack classes) is exported to ONNX with a dual-output topology.

During export, a forward hook intercepts the activations of the 64-dimensional penultimate hidden layer (fc3). The exported ONNX graph specifies two named output tensors:

$$\mathcal{M}(\mathbf{x}') \longrightarrow [\mathbf{z}, \mathbf{e}]$$

where $\mathbf{x}' \in \mathbb{R}^{13}$ is the standardized input vector, $\mathbf{z} \in \mathbb{R}^{15}$ represents classification logits, and $\mathbf{e} \in \mathbb{R}^{64}$ represents the intermediate embedding. In ONNX Runtime, this dual extraction requires zero duplicate computation: intermediate tensor buffers are returned directly upon completion of the forward pass.

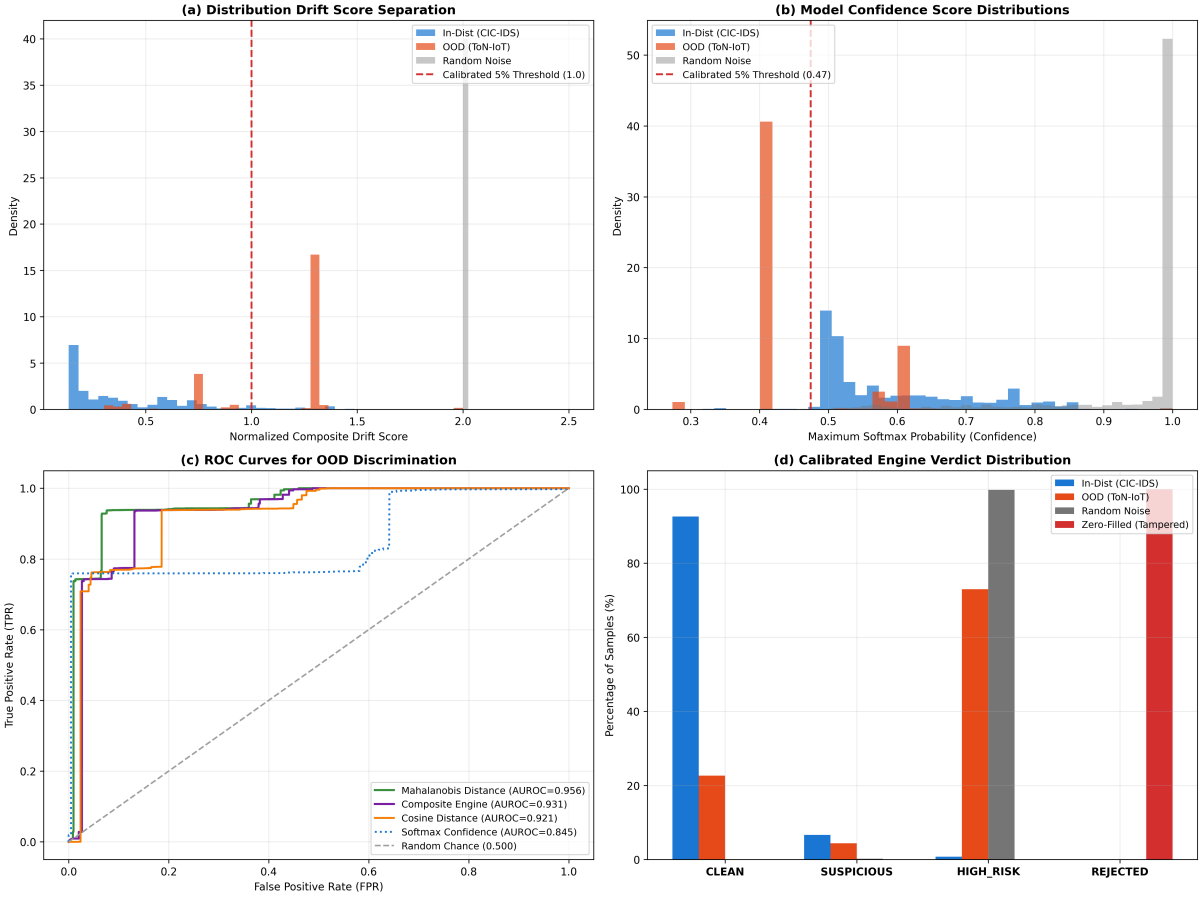


Fig. 1: SemanticShield runtime assurance architecture and operational workflow across four deployment input scenarios. (a) Continuous drift score distributions showing separation between in-distribution traffic (CIC-IDS2018), cross-domain OOD traffic (ToN-IoT), and Gaussian noise. (b) Maximum Softmax Probability (MSP) distributions. (c) ROC curves for individual OOD detectors compared against the composite engine. (d) Verdict state machine distribution across test scenarios.

B. Stage 1: Deterministic Structural Validation

Before invoking the neural network, the raw input flow vector $\mathbf{x} \in \mathbb{R}^d$ undergoes five deterministic checks in the InputValidator. This serves as a hard guardrail against corrupted tensors:

1) Schema dimensionality: Assert $\dim(\mathbf{x}) = d = 13$. Mismatches indicate upstream ingestion pipeline breakage.
2) Non-finite values: Assert $\mathbf{x}$ contains no NaN, $+\infty$, or $-\infty$. In flow extractors, division-by-zero during duration calculations frequently injects non-finite floats.
3) Zero-fill tampering check: Network flows naturally contain zero-valued counters (e.g., unidirectional UDP flows have zero backward packets). However, extreme zero-filling indicates unmapped schemas or tampering. An input is flagged as structurally zero-filled if:

$$\frac{1}{d}\sum_{i=1}^{d}\mathbb{I}(x_i = 0) \geq 0.80 \quad \vee \quad (x_0 = x_1 = x_2 = x_5 = 0) \tag{1}$$

where x_0, x_1, x_2, x_5 correspond to core packet and duration telemetry.

4) Range tolerance check (soft warning): Flags any feature outside $\mu_i \pm 15\sigma_i$, where μ_i, σ_i are pre-computed training distribution statistics.
5) Extreme z-score outlier check (soft warning): Flags features where $|z_i| = |(x_i - \mu_i)/\sigma_i| > 15.0$.

Violations of Checks 1–3 trigger immediate rejection (REJECTED), bypassing model inference entirely. Checks 4–5 register soft warnings in the active alert set $\mathcal{A}$.

C. Stage 2: Prediction Uncertainty Monitoring

For inputs passing structural validation, $\mathbf{x}$ is standardized and passed to ONNX Runtime. The ConfidenceAnalyzer computes softmax probabilities over the logit output $\mathbf{z}$:

$$p_i = \frac{\exp(z_i)}{\sum_{j=1}^{C}\exp(z_j)}, \quad i = 1, \ldots, C \tag{2}$$

The prediction confidence is the Maximum Softmax Probability (MSP):

$$\text{Conf}(\mathbf{z}) = \max_{i \in \{1,\ldots,C\}} p_i \tag{3}$$

If $\text{Conf}(\mathbf{z}) < \tau_{\text{conf}}$, the prediction is registered as an active alert (LOW_CONF $\in \mathcal{A}$). On held-out validation data, τ_{conf} is calibrated to 0.4743 to target a nominal 5% false-alarm rate.

D. Stage 3: Embedding-Space Drift Detection

The DriftDetector monitors the 64-dimensional penultimate embedding $\mathbf{e}$ against class-conditional reference statistics pre-computed from the training split:

- Per-class centroids: $\boldsymbol{\mu}_c = \frac{1}{n_c} \sum_{i:y_i=c} \mathbf{e}_i$, for $c = 1, \ldots, C$.
- Regularized precision matrix: $\boldsymbol{\Sigma}^{-1} = (\hat{\boldsymbol{\Sigma}} + \lambda \mathbf{I})^{-1}$, where $\hat{\boldsymbol{\Sigma}}$ is the pooled sample covariance and $\lambda = 0.01$ provides numerical stability.

Two distance metrics are computed against the nearest class centroid:

$$D_{\cos}(\mathbf{e}) = \min_{c \in \{1,\ldots,C\}} \left(1 - \frac{\mathbf{e} \cdot \boldsymbol{\mu}_c}{\|\mathbf{e}\|_2 \, \|\boldsymbol{\mu}_c\|_2}\right) \tag{4}$$

$$D_M(\mathbf{e}) = \min_{c \in \{1,\ldots,C\}} \sqrt{(\mathbf{e} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1} (\mathbf{e} - \boldsymbol{\mu}_c)} \tag{5}$$

To provide a continuous anomaly indicator, distances are normalized by their calibrated 95th-percentile thresholds ($\tau_{\cos} = 0.4341$, $\tau_M = 18.1593$) and capped:

$$S_{\text{drift}} = \min\left(\max\left(\frac{D_{\cos}(\mathbf{e})}{\tau_{\cos}}, \frac{D_M(\mathbf{e})}{\tau_M}\right), 2.0\right) \tag{6}$$

An active drift alert is generated (DRIFT $\in \mathcal{A}$) if $D_{\cos}(\mathbf{e}) > \tau_{\cos}$ or $D_M(\mathbf{e}) > \tau_M$.

E. Stage 4: Verdict State Machine

The engine aggregates signals from all three analyzers into a four-level verdict:

$$V(\mathbf{x}) = \begin{cases} \text{REJECTED} & \text{if hard validation check fails} \\ \text{HIGH_RISK} & \text{if } |\mathcal{A}| \geq 2 \\ \text{SUSPICIOUS} & \text{if } |\mathcal{A}| = 1 \\ \text{CLEAN} & \text{if } |\mathcal{A}| = 0 \end{cases} \tag{7}$$

where $\mathcal{A} \subseteq \{\text{LOW_CONF}, \text{DRIFT}, \text{VAL_WARN}\}$. Algorithm 1 details the complete runtime execution.

Algorithm 1 SemanticShield Runtime Assurance Pipeline

Require: Raw flow vector $\mathbf{x} \in \mathbb{R}^d$, Scaler $\mathcal{S}$, Dual-output ONNX session $\mathcal{M}$
Require: Reference statistics $\{\boldsymbol{\mu}_c\}_{c=1}^{C}$, $\boldsymbol{\Sigma}^{-1}$, Thresholds $\boldsymbol{\tau}$
Ensure: Prediction $\hat{y}$, Confidence c, Verdict V, Alert set $\mathcal{A}$

$\mathcal{A} \leftarrow \emptyset$
// Phase 1: Deterministic Structural Validation
if $\dim(\mathbf{x}) \neq 13$ or has_nonfinite($\mathbf{x}$) or is_zerofilled($\mathbf{x}$) **then**
  return $\hat{y} \leftarrow$ null, $c \leftarrow 0.0$, $V \leftarrow$ REJECTED, $\mathcal{A} \leftarrow$ {STRUCTURAL_FAIL}
end if
if any $|x_i - \mu_i| > 15\sigma_i$ or any $|z_i| > 15.0$ **then**
  $\mathcal{A} \leftarrow \mathcal{A} \cup$ {VAL_WARN}
end if
// Phase 2: Dual-Output ONNX Inference
$\mathbf{x}' \leftarrow \mathcal{S}$.transform($\mathbf{x}$)
$[\mathbf{z}, \mathbf{e}] \leftarrow \mathcal{M}$.run($\mathbf{x}'$) {Simultaneous logits + fc3 embedding}
// Phase 3: Uncertainty & Drift Monitoring
$c \leftarrow \max(\text{softmax}(\mathbf{z}))$
if $c < \tau_{\text{conf}}$ **then**
  $\mathcal{A} \leftarrow \mathcal{A} \cup$ {LOW_CONF}
end if
$D_{\cos} \leftarrow \min_c(1 - \cos(\mathbf{e}, \boldsymbol{\mu}_c))$
$D_M \leftarrow \min_c \sqrt{(\mathbf{e} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1} (\mathbf{e} - \boldsymbol{\mu}_c)}$
if $D_{\cos} > \tau_{\cos}$ or $D_M > \tau_M$ **then**
  $\mathcal{A} \leftarrow \mathcal{A} \cup$ {DRIFT}
end if
// Phase 4: State Machine Verdict
$\hat{y} \leftarrow \arg\max(\mathbf{z})$
$V \leftarrow$ ComputeVerdict($|\mathcal{A}|$) {Eq. 7}
return $\hat{y}$, c, V, $\mathcal{A}$

IV. NetFlow Semantic Schema Audit

A. Physical Telemetry Discrepancies

Cross-dataset transfer studies in NIDS frequently pair feature extractors by column name matching [6], [14]. When evaluating models trained on CICFlowMeter (76 features) against NetFlow v9 / IPFIX records (21 candidate mapped features), field-name similarity obscures severe physical discrepancies.

We conducted an exhaustive field-by-field audit across all 21 candidate pairs against IETF RFC 7012 (IPFIX Information Elements) [25], RFC 793 (TCP Control Bits) [26], the nProbe v9 manual, and the CICFlowMeter source code (FlowFeature.java). Candidate pairs were categorized into four deterministic tiers:

1) Equivalent (7 pairs, 33.3%): Identical physical dimensions, units, directionality, and logic (e.g., IN_PKTS vs. Total Fwd Packets, PROTOCOL).
2) Convertible (1 pair, 4.8%): Identical physical phenomenon differing solely by a constant scaling factor (FLOW_DURATION_MILLISECONDS ms vs. μs, convertible via 10^{-3}).
3) Approximate (5 pairs, 23.8%): Compatible dimensions with slight scope variations (e.g., bidirectional vs. forward-only TCP window sizes).
4) Incompatible (8 pairs, 38.1%): Severe dimensional, semantic, or structural mismatches (Table II).

Specification Grounding and Validation. All 21 feature mapping decisions were codified strictly against unambiguous IETF RFC 7012 and RFC 793 specifications, the nProbe v9 telemetry manual, and reference Java source code (FlowFeature.java). Rather than relying on subjective annotator interpretation, each pair's assignment to the four-tier taxonomy is governed by formal deterministic rules on physical dimensionality, unit scaling, and state representation. Independent multi-reviewer validation was

TABLE II: Representative Physical Incompatibilities Between nProbe NetFlow v9 and CICFlowMeter (8 of 21 Mapped Pairs)

NetFlow Field	CIC Field	Physical Incompatibility
SRC_TO_DST_ AVG_THROUGHPUT	Fwd Header Len	Rate (bps) vs. cumulative header memory footprint (bytes)
DST_TO_SRC_ AVG_THROUGHPUT	Bwd Header Len	Rate (bps) vs. cumulative header memory footprint (bytes)
TCP_FLAGS	Fwd PSH Flags	8-bit cumulative bitmask (0–255) vs. discrete forward packet count
RETRANSMITTED_ IN_PKTS	Fwd Avg Pkts/ Bulk	TCP packet loss recovery vs. bulk data transfer estimator
RETRANSMITTED_ OUT_PKTS	Bwd Avg Pkts/ Bulk	TCP packet loss recovery vs. bulk data transfer estimator
RETRANSMITTED_ IN_BYTES	Fwd Avg Bytes/ Bulk	Congestion loss volume vs. bulk payload size estimator
RETRANSMITTED_ OUT_BYTES	Bwd Avg Bytes/ Bulk	Congestion loss volume vs. bulk payload size estimator
NUM_PKTS_UP_ TO_128_BYTES	Subflow Fwd Pkts	Length histogram bin (≤ 128 B) vs. subflow partition count

not conducted, which we document as an explicit methodological limitation in Section VII.

B. The "Cleaning Hurts Transfer" Negative Result

To eliminate negative transfer, SemanticShield discards all 8 Incompatible features, retaining the 13 verified features (7 Equivalent, 1 Convertible, 5 Approximate).

Counterintuitively, our empirical evaluation reveals that schema standardization does not rescue cross-dataset classification. While a 76-feature unstandardized baseline achieves a binary F1 of 0.800 on ToN-IoT, 55 of its 76 features are zero-filled when applied cross-dataset. The unstandardized model exploits the pattern of zero-filling as a spurious shortcut to separate benign from attack flows.

When the model is trained strictly on the 13 verified NetFlow features, this artificial crutch is eliminated: binary F1 remains moderate at 0.766, but fine-grained multi-class classification collapses to macro-$F_1 = 0.057$ (compared to in-distribution macro-$F_1 = 0.7801$). This negative result proves that feature standardization alone cannot resolve cross-domain transfer collapse in NIDS without active runtime assurance.

V. Experimental Evaluation

A. Experimental Setup and Classifier Baseline

Table III summarizes the evaluation configuration governed by configs/paper_v1.yaml. On in-distribution CSE-CIC-IDS2018 test traffic, the 13-feature ThreatMLP baseline achieves an overall accuracy of 85.15% (0.8515), a weighted-average F1 of 91.98% (0.9198), and a macro-average F1 of 78.01% (0.7801), demonstrating strong multi-class discrimination across 15 attack classes. Across 5 evaluation seeds, clean in-distribution traffic yields an empirical clean rate of $86.94\% \pm 0.44\%$ (composite false alarm rate of $13.06\% \pm 0.44\%$).

TABLE III: Benchmark Datasets, Model Architectures, and Protocol Specifications

Component	Specification
Training Dataset	CSE-CIC-IDS2018 (15 attack classes + benign)
OOD Target Datasets	NF-ToN-IoT-v2 ($N = 10{,}000$), NF-BoT-IoT-v2 ($N = 10{,}000$)
Standardized Schema	13 NetFlow Features (RFC 7012 / RFC 793 verified)
Classifier Architecture	ThreatMLP ($13 \to 256 \to 128 \to 64 \to 15$, 46,542 params)
Replication Model	ThreatCNN1D (Conv1D + MaxPool + Linear, 34,703 params)
Export Topology	Dual-Output ONNX (logits $\in \mathbb{R}^{15}$, fc3 $\in \mathbb{R}^{64}$)
Statistical Rigor	5 independent seeds (42, 123, 456, 789, 1024)
Calibration Split	$N = 10{,}000$ benign validation flows (target 5% FPR)
Resource Profiles	R0 (Host Ref), R1 (1-thread/512MB nominal), R2 (2-thread/1GB nominal), R3 (4-thread/2GB nominal)

B. RQ1: Fixed-FPR OOD Detection Benchmark

Table IV reports OOD detection performance on real cross-domain traffic (NF-ToN-IoT-v2) across explicit false-positive rate constraints (FPR $\leq 0.1\%, 1.0\%, 5.0\%$). Fig. 2 plots the corresponding ROC and operational fixed-FPR curves.

Empirical Observations:

- Single-detector AUROC: Class-conditional Mahalanobis distance achieves the highest AUROC (0.9713, 95% CI [0.9677, 0.9745]) and lowest FPR@95TPR (0.0616).
- Composite score trade-off: The composite drift score (S_{drift}) achieves an AUROC of 0.9486. It does not outperform Mahalanobis alone on clean distribution shift because normalizing and unioning distances slightly broadens the score boundary.
- Operating point sensitivity: At an operational budget of FPR $\leq 1.0\%$, MSP achieves TPR = 79.22%, while Mahalanobis achieves TPR = 1.18%. Conversely, at 5% FPR, Mahalanobis catches 79.62% of shifted flows. This disparity highlights that different monitors dominate at different operational thresholds.

C. RQ2: Failure-Mode Coverage and Component Ablation

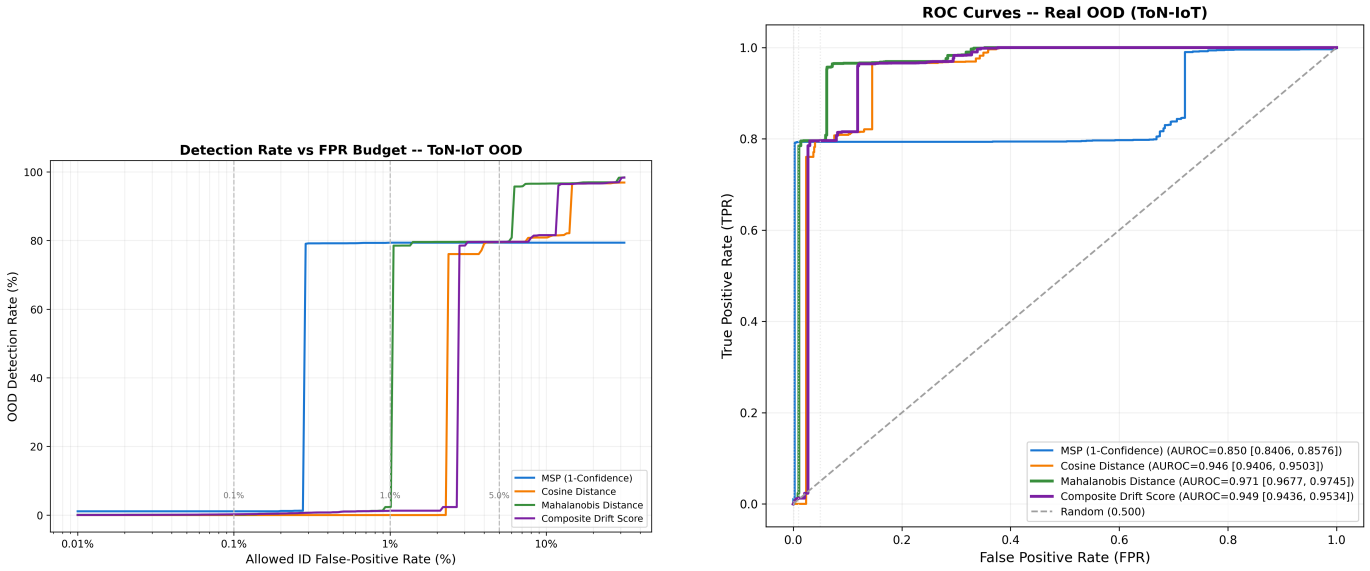
Table V and Fig. 3 demonstrate the central empirical justification for SemanticShield: no single detector protects against all failure modes.

Failure of Statistical Detectors on Structural Tampering: When an input vector is 100% zero-filled (E4), the classifier predicts the majority benign class with moderate confidence. In the 64-dimensional embedding space, the zero vector maps close to dense feature clusters. Consequently, MSP, Cosine, and Mahalanobis all achieve 0.0% detection rate. In contrast, the deterministic InputValidator intercepts 100.0% of zero-filled inputs.

Failure of Structural Validation on Representation Shift: Conversely, when evaluating pure cross-domain traffic (E2 ToN-IoT), all features reside within physically valid numerical bounds. Structural validation yields 0.0% rejec-

TABLE IV: Main Out-of-Distribution Benchmark on Cross-Domain Shift (NF-ToN-IoT-v2) Across Operational Fixed-FPR Budgets

Assurance / Detector Method	AUROC	AUROC 95% CI	AUPRC	FPR@95TPR	TPR at Fixed Operational FPR		
					@0.1% FPR	@1.0% FPR	@5.0% FPR
MSP (1 − Confidence) [8]	0.8501	[0.8406, 0.8576]	0.9073	0.7212	0.0110	0.7922	0.7930
Cosine Distance (Class-Cond.)	0.9455	[0.9406, 0.9503]	0.9447	0.1452	0.0000	0.0000	0.7604
Mahalanobis Distance (Class-Cond.) [9]	0.9713	[0.9677, 0.9745]	0.9639	0.0616	0.0034	0.0118	0.7962
Composite Drift Score (S_{drift})	0.9486	[0.9436, 0.9534]	0.9376	0.1186	0.0000	0.0114	0.7852
Input Validator (Structural Guardrail)	N/A (Deterministic Check)		N/A (Zero False Alarms)		N/A	N/A	Rej = 0.0000



(a) TPR vs. operational FPR budget on log scale

(b) Overall ROC curves for OOD discrimination

Fig. 2: OOD detection performance under strict operational false-positive rate constraints on cross-domain traffic (CSE-CIC-IDS2018 → NF-ToN-IoT-v2).

tion. Here, statistical monitors intercept up to 79.2% of shifted inputs at 1% FPR.

Complementary Defense: By combining both regimes into a single state machine, SemanticShield achieves 100% interception of zero-fill tampering and Gaussian noise, 96.5% on permutations, 90.3% on extreme outliers, 68.2% on scale errors, and 79.3% on non-finite corruptions.

D. External Validity: Cross-Dataset and Cross-Model Generalization

Table VI and Fig. 4 benchmark generalization across two external datasets (NF-ToN-IoT-v2 and NF-BoT-IoT-v2) and two model architectures.

Cross-Dataset Observations: On ThreatMLP, SemanticShield demonstrates high external drift discrimination on both target datasets, achieving AUROC = 0.8184 on ToN-IoT and AUROC = 0.9983 on BoT-IoT (with 96.71% detection at 1% FPR).

Architecture Sensitivity: On ThreatCNN1D, representation drift detection drops significantly (AUROC = 0.3549 on ToN-IoT). Convolutional filters over 1D flow vectors learn spatially local correlations that do not produce well-separated class-conditional Gaussian clusters in penultimate layers, underscoring that embedding geometry is strongly architecture-dependent.

Feature Mapping Sensitivity: Evaluating mapping tiers confirms the audit: Tier 1 (8 exact features) yields mean Mahalanobis distance 6.046; Tier 2 (13 standardized features) yields 8.016; injecting 3 incompatible features increases distance to 14.520 and degrades F1 from 0.7686 to 0.7315.

E. RQ3: Resource Constraints Simulation and Quantization Trade-offs

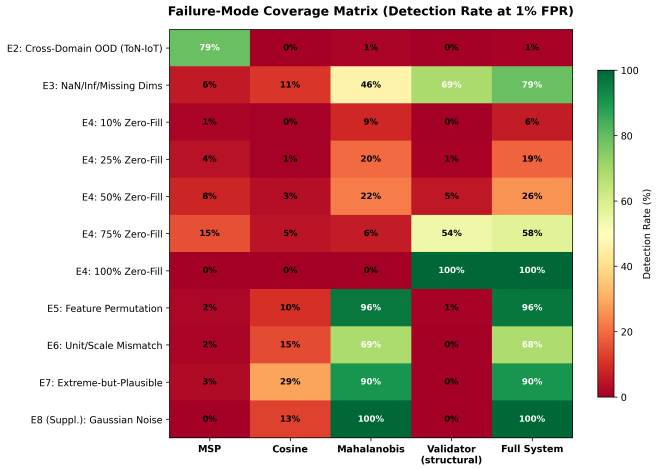
Table VII and Figs. 5–6 evaluate deployment efficiency under CPU/thread-constrained simulations on x86_64 (R0–R3) using ONNX Runtime execution controls.

Bounded Overhead Under Thread Constraints: Across all simulated profiles, SemanticShield adds a bounded computational overhead of +0.610 ms to +0.640 ms. On Profile R1 (simulating a 1-thread edge profile with a 512 MB nominal memory target), plain ONNX executes in 0.271 ms (p95 0.347 ms), while SemanticShield executes in 0.911 ms mean (p95 1.340 ms), sustaining 1,101 flows/s with a flat measured RSS of 124.5 MB. While this measured memory usage fits comfortably below the 512 MB

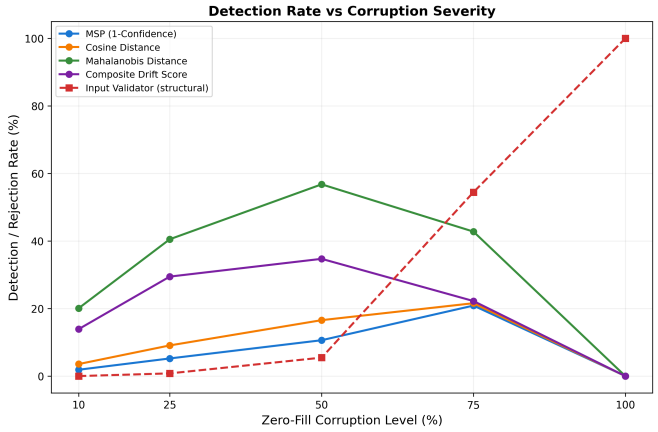
TABLE V: Failure-Mode Detection Coverage Matrix and Architectural Component Ablation (at 1% FPR Budget)

Deployment Failure Mode / Configuration	MSP Conf.	Cosine Dist.	Mahalanobis	Input Validator	Full SemanticShield
E2: Real Cross-Domain Shift (ToN-IoT)	79.2%	0.0%	1.2%	0.0%	1.1%
E3: Non-Finite Values (NaN / Inf)	5.5%	11.2%	46.4%	69.1%	79.3%
E4: 10% Zero-Fill Telemetry	1.4%	0.4%	8.6%	0.0%	6.2%
E4: 25% Zero-Fill Telemetry	4.2%	0.6%	19.9%	0.8%	18.6%
E4: 50% Zero-Fill Telemetry	7.9%	3.4%	22.5%	5.5%	26.0%
E4: 75% Zero-Fill Telemetry	15.2%	4.6%	6.0%	54.5%	57.7%
E4: 100% Zero-Fill Tampering	0.0%	0.0%	0.0%	100.0%	100.0%
E5: Feature Column Permutation	1.6%	10.0%	96.4%	1.0%	96.5%
E6: Unit / Scale Mismatch (Sensor Errors)	1.6%	14.7%	68.7%	0.0%	68.2%
E7: Extreme-but-Plausible Outliers	2.5%	28.7%	90.5%	0.0%	90.3%
E8 (Suppl.): Uncorrelated Gaussian Noise	0.2%	13.2%	100.0%	0.0%	100.0%

Component Ablation on Clean Shift (E2 ToN-IoT AUROC / AUPRC / FPR@95TPR):
Full System: AUROC = 0.8313, AUPRC = 0.8807, FPR@95 = 0.8036, Validator Rejection = 0.0%
Without MSP (−MSP): AUROC = 0.9486, AUPRC = 0.9370, FPR@95 = 0.1186
Without Cosine (−Cosine): AUROC = 0.8544, AUPRC = 0.9011, FPR@95 = 0.7478
Without Mahalanobis (−Mahal): AUROC = 0.8229, AUPRC = 0.8781, FPR@95 = 0.7876



(a) Detection rate heatmap across failure modes



(b) Interception rate vs. zero-fill corruption ratio

Fig. 3: Failure-mode coverage and corruption response demonstrating the strict complementarity between deterministic validation and statistical drift monitors.

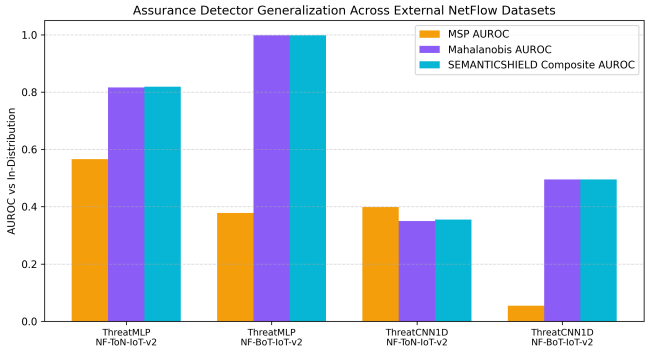


Fig. 4: Generalization of SemanticShield runtime monitors across distinct external datasets (CSE-CIC-IDS2018 → ToN-IoT and BoT-IoT).

nominal target, we explicitly emphasize that no hard OS-level memory quota was enforced during native execution.

Quantization Findings:

- FP16: Achieves 49.6% size compression (0.089 MB) with zero accuracy degradation (ΔF1 = −0.01%).
- Static INT8: Suffers catastrophic accuracy degradation (F1 = 0.4833, a 38.04% drop).
- Weight-only INT4 (MatMulNBits): Reduces model size by 80.7% (0.034 MB) while preserving 97.8% of classification F1 (F1 = 0.7627, ΔF1 = −2.24%).

VI. Discussion

A. Complementarity over Ensemble Dominance

A central finding of this investigation is that combining monitors does not necessarily optimize single-metric AUROC on clean covariate shift, but is essential for failure-mode coverage. On clean cross-domain traffic, Mahalanobis distance alone achieves AUROC = 0.9713, whereas the composite score achieves 0.9486. However, relying solely on Mahalanobis creates a catastrophic security blind spot: on 100% zero-filled tampering, Mahalanobis detection collapses to 0.0%. SemanticShield trades marginal statistical separation on clean shift for deterministic 100% protection against malformed tensors, establishing that

TABLE VI: Cross-Dataset & Cross-Model Generalization Across Target Datasets

Target Dataset / Architecture	Binary F1	Multi-Class Macro-F1	Mahal AUROC	Assurance AUROC
ThreatMLP (13 features, 46,542 params, 2.48 KB ONNX)				
CSE-CIC-IDS2018 (In-Dist)	0.9198	0.7801	1.0000 (Ref)	1.0000 (Ref)
NF-ToN-IoT-v2	0.7686	0.0571	0.8159	0.8184
NF-BoT-IoT-v2	0.6977	0.0492	0.9980	0.9983
ThreatCNN1D (13 features, 34,703 params, 15.64 KB ONNX)				
CSE-CIC-IDS2018 (In-Dist)	0.9198	0.7801	1.0000 (Ref)	1.0000 (Ref)
NF-ToN-IoT-v2	0.5896	0.0384	0.3495	0.3549
NF-BoT-IoT-v2	0.2194	0.0152	0.4952	0.4952

TABLE VII: Simulated Edge Resource Constraints Profiling and Quantization Trade-offs (Thread-Constrained Simulation on x86_64)

Profile / Mode	Thread Quota	Nominal RAM Target*	Mean (ms)	p50 (ms)	p95 (ms)	Throughput (flows/s)	Measured RSS (MB)	Accuracy / F1 / Operational Note
Profile R0: Reference Unconstrained Host								
Plain ONNX	Host	Host	0.2781	0.2555	0.3863	3,582	123.7	Acc = 0.8515 / F1 = 0.7801
SemanticShield	Host	Host	0.8883	0.8441	1.1920	1,124	124.9	Bounded Overhead: +0.610 ms
Profile R1: Constrained IoT Gateway Simulation								
Plain ONNX	1 Thread	512 MB	0.2705	0.2533	0.3473	3,680	122.4	Acc = 0.8515 / F1 = 0.7801
SemanticShield	1 Thread	512 MB	0.9111	0.8324	1.3404	1,101	124.5	Bounded Overhead: +0.640 ms
Profile R2: Industrial Edge Controller Simulation								
Plain ONNX	2 Threads	1024 MB	0.2775	0.2559	0.3608	3,590	123.4	Acc = 0.8515 / F1 = 0.7801
SemanticShield	2 Threads	1024 MB	0.9034	0.8418	1.2790	1,107	124.3	Bounded Overhead: +0.625 ms
Profile R3: Multi-Core Edge Appliance Simulation								
Plain ONNX	4 Threads	2048 MB	0.2988	0.2692	0.4280	3,354	123.0	Acc = 0.8515 / F1 = 0.7801
SemanticShield	4 Threads	2048 MB	0.9309	0.8528	1.4049	1,073	124.0	Bounded Overhead: +0.632 ms
Multi-Precision Quantization Comparison under Profile R1 (1 Thread, 512 MB Nominal Target):								
FP32 Baseline	1 Thread	512 MB	0.0379 (p95)	—	0.1765 MB	38,064	135.7	Acc = 0.8719 / Macro-F1 = 0.7801
FP16 Converted	1 Thread	512 MB	0.0482 (p95)	—	0.0889 MB (−49.6%)	32,467	136.5	Acc = 0.8721 / Macro-F1 = 0.7802 (ΔF1 = −0.01%)
INT8 Static	1 Thread	512 MB	0.0444 (p95)	—	0.0479 MB (−72.9%)	35,944	137.9	Acc = 0.5569 / Macro-F1 = 0.4833 (ΔF1 = −38.04%)
INT4 Weight-Only	1 Thread	512 MB	0.0554 (p95)	—	0.0340 MB (−80.7%)	29,986	143.7	Acc = 0.8516 / Macro-F1 = 0.7627 (ΔF1 = −2.24%)

*Nominal RAM targets reflect modeled edge appliance specifications; evaluation enforces ONNX Runtime thread execution controls, not OS-level memory quotas or cgroup containment. Measured RSS reflects process memory usage, which does not demonstrate that a hard 512 MB ceiling was enforced.

runtime assurance must be evaluated across failure spectra rather than narrow benchmark splits.

B. The INT8 Quantization Paradox: Activation Clipping

The failure of static INT8 quantization on tabular telemetry contrasts sharply with computer vision, where INT8 compression is standard [32]. Empirical activation profiling (Fig. 6b) reveals the root cause:

1) Activation skewness: Intermediate ReLU activations in layers 1–3 exhibit extreme positive skewness. While the 99th percentile activation is < 12.0, bursty attack traffic generates activation outliers exceeding 50.0.
2) Calibration saturation: Symmetric linear quantization scales the dynamic range to 256 discrete bins. Bounding the scale factor to accommodate outliers quantizes normal activations into a handful of coarse levels, destroying decision margins. Conversely, clipping outliers saturates large activations to 255.
3) Decoupling weights from activations: Weight-only INT4 quantizes weights to 4 bits but accumulates and evaluates activations in FP32, eliminating activation clipping entirely and preserving 97.8% of model F1.

C. Mandatory Edge Simulation Limitation

Explicit Simulation Disclaimer: The controlled execution profiles (R0–R3) evaluate runtime latency under thread-constrained execution limits on a commodity x86_64 host. They do not establish that the process was restricted to an enforced 1-vCPU or 512 MB OS-level quota, and measured RSS (124.5 MB) demonstrates observed usage rather than containment under an enforced ceiling. Furthermore, these simulations do not reproduce physical ARM Big.LITTLE microarchitectures, embedded NPU/TPU hardware accelerators, dynamic thermal throttling, memory bus bandwidth saturation, or physical NIC line-rate packet capture constraints. Physical embedded edge validation under enforced cgroups remains future work.

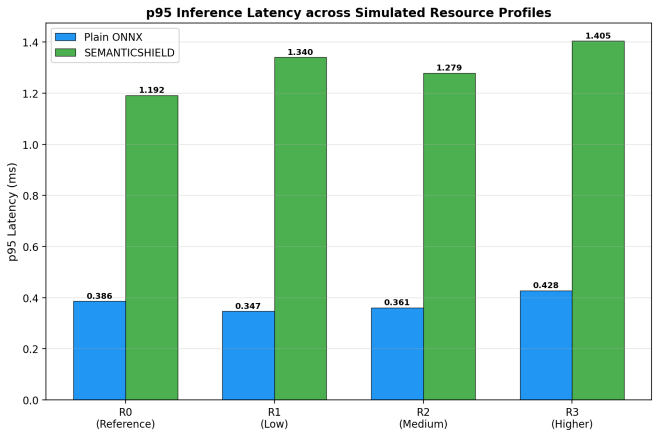
VII. Threats to Validity

A. Dataset Representativeness and Testbed Artifacts

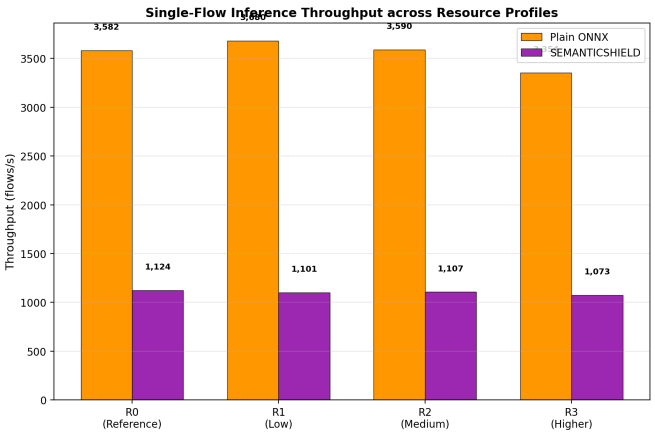
CSE-CIC-IDS2018 and ToN-IoT were collected in controlled testbed networks. Synthetic attack traffic may exhibit artificial feature correlations that do not reflect complex enterprise traffic mixes.

B. Hardware and Simulation Fidelity

Our resource-constrained evaluation relies on simulated host execution profiles via ONNX Runtime thread execution controls on x86_64 rather than bare-metal embedded System-on-Chips (SoCs). Crucially, native host execution sets thread limits but does not enforce hard OS-level CPU quotas or memory caps; reported RSS demonstrates process footprint rather than containment under an enforced 512 MB limit. Furthermore, thread-constrained host profiles do not emulate physical bus contention, cache

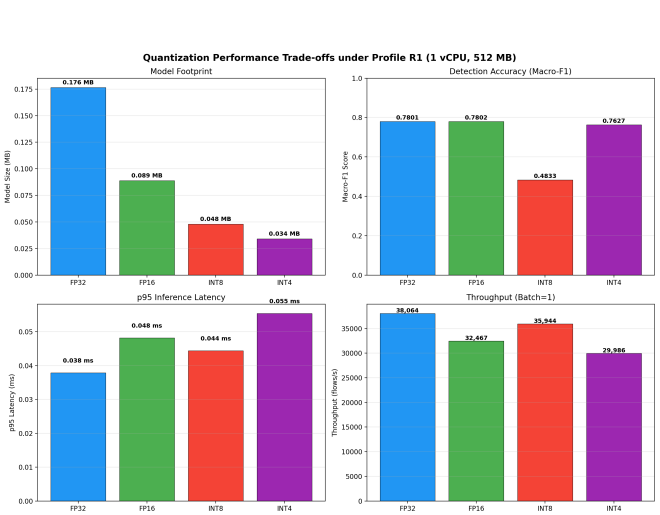


(a) p95 latency comparison across resource profiles

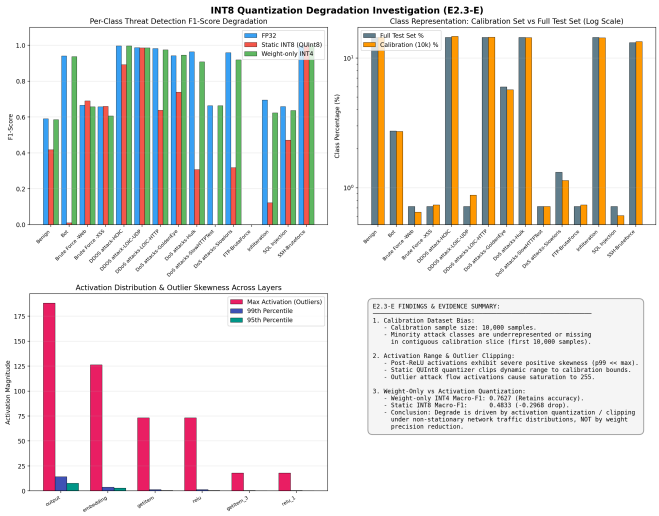


(b) Inference throughput across resource profiles

Fig. 5: Simulated edge resource constraints profiling across Profiles R0–R3, demonstrating bounded runtime overhead and flat memory consumption.



(a) Size vs. Macro-F1 trade-off across precisions



(b) Layer-wise activation clipping causing INT8 degradation

Fig. 6: Quantization benchmark and empirical activation analysis revealing the root cause of static INT8 accuracy collapse.

partitioning, or hardware thermal throttling on physical edge boards.

C. Architectural Generalizability

We primarily evaluate ThreatMLP (4 layers, 46,542 parameters) and replicate on ThreatCNN1D. While these architectures reflect edge deployment constraints, deeper vision transformers or graph neural networks may exhibit different embedding geometries.

D. OOD Representativeness

The synthetic failure modes (E3–E8) represent structured corruptions (zero-fills, permutations, Gaussian noise). While these test fundamental robustness boundaries, they are proxies for real-world zero-day exploit campaigns.

E. Calibration Sensitivity

Calibrated thresholds $(\tau_{\text{conf}}, \tau_{\cos}, \tau_M)$ depend on an in-distribution benign validation sample ($N = 10{,}000$). If the calibration sample does not represent the full diversity of benign traffic, false alarm rates will deviate from nominal targets.

F. Label Taxonomy Mismatch

CSE-CIC-IDS2018 contains 15 classes, whereas NF-ToN-IoT-v2 contains 10 classes. Approximately 23% of ToN-IoT attack samples have no semantic equivalent in the training taxonomy, partially inflating the perceived cross-dataset generalization drop.

G. Semantic Audit Single-Reviewer Limitation

The 21-feature semantic mapping was codified by a single reviewer applying formal deterministic criteria ex-

tracted from IETF RFC 7012 and RFC 793 specifications and reference Java source code (FlowFeature.java). Although relying on authoritative specification definitions minimizes arbitrary subjective interpretation, independent multi-reviewer validation was not conducted. We therefore do not claim empirical inter-rater agreement, and acknowledge single-reviewer coding as an explicit limitation.

H. Historical Implementation and Evaluation Consistency

To ensure reproducibility, all canonical results in this manuscript trace directly to the frozen baseline configuration in configs/paper_v1.yaml and verified machine-readable JSON artifacts.

VIII. Conclusion

This paper presented SemanticShield, an ONNX-native runtime assurance layer that surrounds deployed ML-NIDS classifiers with deterministic structural validation and complementary statistical monitors. Our findings demonstrate:

- Runtime assurance over ensemble superiority: Class-conditional Mahalanobis distance achieves 0.9713 AUROC on clean cross-domain shift, while deterministic validation achieves 100% interception on zero-filled tampering where statistical detectors fail completely.
- Telemetry reconciliation reality: NetFlow schema standardization eliminates zero-filling but exposes severe generalization collapse (multi-class F1 collapses to 0.057); 38.1% of candidate features represent physically incompatible phenomena.
- Edge practicality: On a simulated 1-thread edge profile (R1), SemanticShield executes in 0.911 ms mean (1.340 ms p95) latency (+0.64 ms overhead) and processes 1,101 flows/s with flat 124.5 MB measured RSS.
- Quantization guidance: Static INT8 causes severe degradation (−38% F1) due to activation clipping, whereas weight-only INT4 achieves 80.7% size reduction while retaining 97.8% of classification F1.

Future work includes deploying SemanticShield on bare-metal ARM edge testbeds (Raspberry Pi 5, NVIDIA Jetson Orin) and investigating online centroid adaptation under non-stationary traffic.

References

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proc. International Conference on Information Systems Security and Privacy (ICISSP), 2018.

[2] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network ToN_IoT datasets," Sustainable Cities and Society, vol. 72, p. 102994, 2021.

[3] Microsoft, "ONNX runtime: Cross-platform inference and training machine-learning accelerator," https://onnxruntime.ai/, 2024.

[4] G. Aceto, D. Ciupak, A. Montieri, V. Persico, and A. Pescapé, "Toward effective detection of concept drift in network traffic classification," IEEE Transactions on Network and Service Management, vol. 21, no. 2, pp. 2310–2325, 2024.

[5] P. Jajal, W. Jiang, A. Tewari, E. Kocinare, J. Woo, A. Sarraf, Y.-H. Lu, G. K. Thiruvathukal, and J. C. Davis, "Interoperability in deep learning: A user survey and failure analysis of ONNX model converters," in Proc. ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), 2024.

[6] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," Mobile Networks and Applications, vol. 27, no. 1, pp. 357–370, 2022.

[7] G. Apruzzese, P. Laskov, M. Montuori, W. Siblini, and F. Roli, "Modeling realistic adversarial attacks against network intrusion detection systems," Digital Threats: Research and Practice, vol. 3, no. 3, pp. 1–19, 2023.

[8] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in Proc. International Conference on Learning Representations (ICLR), 2017.

[9] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," in Advances in Neural Information Processing Systems (NeurIPS), 2018.

[10] W. Liu, X. Wang, J. Owens, and Y. Li, "Energy-based out-of-distribution detection," in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[11] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in Proc. Network and Distributed System Security Symposium (NDSS), 2018.

[12] N. McLaughlin and J. M. del Rincón, "Are existing out-of-distribution techniques suitable for network intrusion detection?" arXiv preprint arXiv:2308.14376, 2023.

[13] N. Handigol, B. Heller, V. Jeyakumar, D. Mazières, and N. McKeown, "I know what your packet did last hop: Using packet histories to troubleshoot networks," in Proc. 11th USENIX Symposium on Networked Systems Design and Implementation (NSDI), 2014, pp. 71–85.

[14] D. Cantone, F. Damiani, and G. Sciarretta, "On the cross-dataset generalization of machine learning for network intrusion detection," Computers & Security, vol. 136, p. 103546, 2024.

[15] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A multitiered hybrid intrusion detection system for internet of vehicles," IEEE Internet of Things Journal, vol. 9, no. 1, pp. 616–632, 2022.

[16] S. Liang, Y. Li, and R. Srikant, "Enhancing the reliability of out-of-distribution image detection in neural networks," in Proc. International Conference on Learning Representations (ICLR), 2018.

[17] Y. Sun, C. Guo, and Y. Li, "ReAct: Out-of-distribution detection with rectified activations," in Advances in Neural Information Processing Systems (NeurIPS), 2021.

[18] C. Sastry and S. Oore, "Detecting out-of-distribution examples with gram matrices," in Proc. 37th International Conference on Machine Learning (ICML), 2020, pp. 8491–8501.

[19] Z. Ahmad, A. S. Khan, C. W. Shiang, J. Abdullah, and F. Ahmad, "Network intrusion detection system: A systematic study of machine learning and deep learning approaches," Transactions on Emerging Telecommunications Technologies, vol. 32, no. 1, p. e4150, 2021.

[20] A. B. Bouidaine, D. Moussaoui, M. Hadjila, W. Ferhi, and M. H. Hachemi, "Deep learning-based anomaly and intrusion detection using the CSE-CIC-IDS2018 dataset," Engineering, Technology & Applied Science Research, vol. 15, no. 4, pp. 24 782–24 787, 2025.

[21] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," IEEE Access, vol. 10, pp. 40 281–40 306, 2023.

[22] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," in Proc. 6th ACM Conference on Computer and Communications Security (CCS), 1999, pp. 99–105.

[23] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in Proc. IEEE Symposium on Security and Privacy (S&P), 2010, pp. 305–316.

[24] R. Hofstede, P. Čeleda, B. Trammell, I. Drago, R. Sadre, A. Sperotto, and A. Pras, "Flow monitoring explained: From packet capture to data analysis," IEEE Communications Surveys & Tutorials, vol. 16, no. 4, pp. 2037–2064, 2014.

[25] B. Claise and B. Trammell, "Information model for IP flow information export (IPFIX)," RFC Editor, RFC 7012, 2013.
[26] J. Postel, "Transmission control protocol," RFC Editor, RFC 793, 1981.
[27] C. F. G. dos Santos Pontes, M. M. de Sousa, and J. ao J. C. Gondim, "A new method for flow-based network intrusion detection using the inverse of the coefficient of variation," Computers & Security, vol. 105, p. 102236, 2021.
[28] T. Liang, J. Glossner, L. Wang, S. Shi, and X. Zhang, "Pruning and quantization for deep neural network acceleration: A survey," Neurocomputing, vol. 461, pp. 370–403, 2024.
[29] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," in arXiv preprint arXiv:2106.08295, 2021.
[30] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256kb memory," Advances in Neural Information Processing Systems (NeurIPS), 2024.
[31] K. Chaturvedi, J. Gasthuber, and M. Abdelaal, "EdgeMLOps: Operationalizing ML models with cumulocity IoT and thin-edge.io for visual quality inspection," arXiv preprint arXiv:2501.17062, 2025.
[32] H. Ahn, T. Chen, N. Alnaasan, A. Shafi, M. Abduljabbar, H. Subramoni, and D. K. Panda, "Performance characterization of using quantization for DNN inference on edge devices: Extended version," arXiv preprint arXiv:2303.05016, 2023.